



Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório do Trabalho Grupo de Inteligência Artificial para as Telecomunicações

Otimização de Gestão de Veículos Autónomos em Centros de Distribuição

Trabalho realizado pelo grupo 3:

PG61434 Luis Baptista DELTA = 0

PG59972 Bernardo Salgado DELTA = 0

PG58106 Hélder Freitas DELTA = 0

Declaração de Integridade Académica:

Os elementos deste grupo declaram ter atuado com integridade académica e não ter recorrido a práticas de plágio nem a qualquer forma de utilização indevida de informação ou falsificação de resultados. Declaram conhecer e respeitar o Código de Conduta Ética da Universidade do Minho.

2025/2026

Índice

1. Introdução	5
1.1. Contexto e Objetivos	5
1.2. Metodologia.....	5
2. Formulação e Modelação do Problema	6
2.1. Descrição do Cenário LogiBot	6
2.2. Formulação	6
2.3 Função de Custo	7
3. Arquitetura E Algoritmos De Procura	8
3.1. Representação em Grafo.....	8
3.2. Algoritmos Implementados	8
3.3. Heurísticas (Algoritmos Informados).....	8
4. Algoritmos De Procura Implementados	9
4.1. Algoritmos Não-Informados.....	9
4.1.1. Depth-First Search (DFS).....	9
4.1.2. Breadth-First Search (BFS)	9
4.2 Algoritmos Informados	9
4.2.1 Heurística Utilizada	9
4.2.2. Greedy Search (Procura Gulosa)	10
4.2.3. A* (A-Star)	10
4.3. Algoritmos Extra	10
4.3.1. Dijkstra (Custo Uniforme).....	10
4.3.2. DFS com Profundidade Limitada	11
5. Cenários Dinâmicos.....	11
5.1. Mecanismo de Eventos Temporais	11
5.2. Rerouting Automático	11
5.3. Priorização de Encomendas.....	11
6. Resultados Experimentais	12
6.1. Cenário 1 (Validação em Grafo Simples).....	12
6.2. Cenário 2 (Centro Logístico).....	12
6.3. Cenário 3 (Testes Dinâmicos).....	13
7. Análise E Discussão De Resultados	15
7.1. Comparação de Algoritmos	15

7.2. Eficiência Computacional.....	15
8. Conclusão	16
9. Referências	17

Índice de Figuras

Figura 1 - Grafo de Aplicação - Logibot	6
Figura 2 - Teste do Cenário 3	14

Índice de Tabelas

Tabela 1 - Resultados do Cenário 1	12
Tabela 2 - Comparação de Algoritmos no Cenário LogiBot	13

1. Introdução

1.1. Contexto e Objetivos

A gestão logística moderna enfrenta desafios sem precedentes devido ao crescimento exponencial do comércio eletrónico e à densificação dos centros urbanos. A eficiência no transporte de mercadorias deixou de ser apenas uma questão financeira para se tornar um imperativo ambiental e operacional. Neste cenário, a utilização de Veículos Autónomos (VAs) em centros de distribuição, como o proposto para a empresa LogiBot, apresenta-se como uma solução viável para automatizar processos, reduzir erros humanos e otimizar o consumo energético.

O problema central abordado neste projeto consiste no planeamento de trajetórias para múltiplos agentes num ambiente controlado, mas dinâmico. O sistema deve ser capaz de calcular rotas livres de colisão, respeitar janelas temporais de entrega e adaptar-se a imprevistos, como avarias ou bloqueios de corredores. A relevância académica deste trabalho reside na aplicação prática de algoritmos de Inteligência Artificial (IA) clássica, especificamente algoritmos de procura em espaços de estados, para resolver problemas de otimização no mundo real.

1.2. Metodologia

O objetivo principal foi o desenvolvimento de um sistema de software capaz de gerir a navegação de veículos no centro de distribuição LogiBot. Este objetivo desdobra-se em metas específicas:

1. **Modelação do Ambiente:** Representar a infraestrutura física (docas, armazéns, zonas de carga) sob a forma de um grafo ponderado, onde os custos das arestas refletem distâncias ou tempos de trânsito.
2. **Implementação de Estratégias de Procura:** Desenvolver e comparar algoritmos de procura não-informada (DFS, BFS) e informada (Greedy, A*), além de algoritmos de otimização de caminho (Dijkstra).
3. **Gestão de Dinamismo:** Implementar mecanismos que permitam ao sistema reagir a alterações em tempo real, como bloqueios de rotas para manutenção ou a chegada de encomendas prioritárias.
4. **Análise Comparativa:** Avaliar quantitativamente o desempenho dos algoritmos em termos de completude, otimalidade, complexidade temporal e espacial.

2. Formulação e Modelação do Problema

2.1. Descrição do Cenário LogiBot

O ambiente operacional da LogiBot é um centro de distribuição composto por diversas localizações estratégicas. Para efeitos de simulação, o espaço foi dividido num conjunto finito de nós e ligações. As localizações principais identificadas e modeladas são:

- **DEPOT:** Ponto de partida e chegada dos veículos, onde também se realizam recargas de bateria.
- **ZONAS DE DISTRIBUIÇÃO (A e B):** Áreas intermediárias que facilitam o encaminhamento de tráfego.
- **CLIENTES (1, 2 e 3):** Pontos finais de entrega que simulam a saída de encomendas do sistema.
- **ARMAZÉNS:** Locais de armazenamento temporário ou destino final interno.

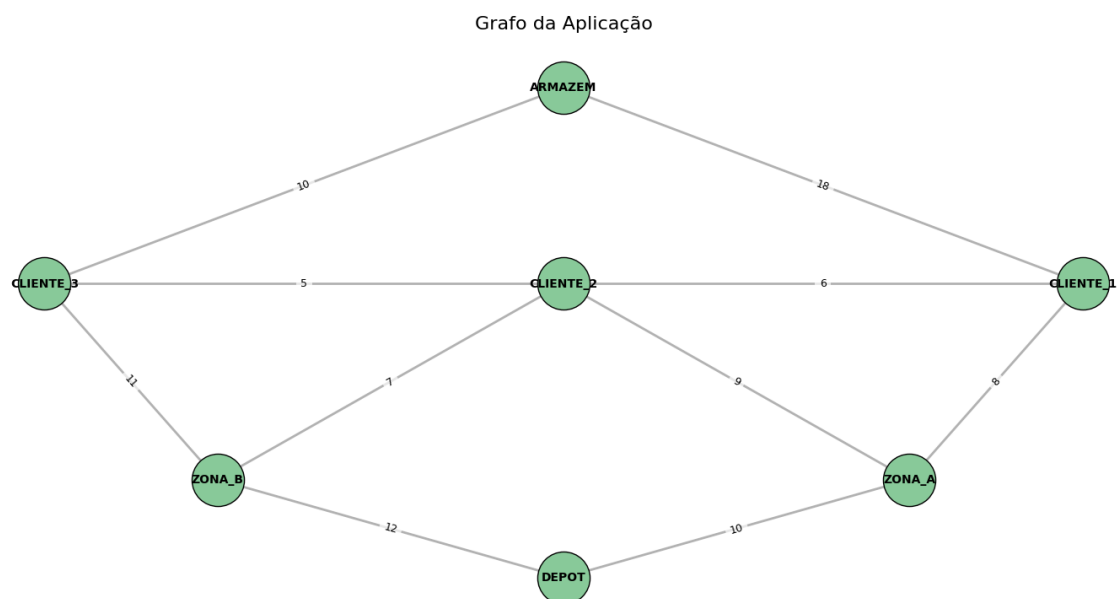


Figura 1 - Grafo de Aplicação - Logibot

2.2. Formulação

O problema foi formulado como um problema de procura em espaço de estados, definido formalmente pela tupla $\langle S, S_0, A, T, C \rangle$:

1. **Espaço de Estados (S):** O estado do sistema num dado instante t é representado pela configuração de todos os veículos e encomendas.

- $St = Pos(v_i), Carga(v_i), Bat(v_i), Status(e_j)$, onde Pos é a localização atual do veículo, $Carga$ é o peso transportado, Bat é o nível de energia e $Status$ indica se a encomenda e_j está "Pendente", "Em Trânsito" ou "Entregue".

2. Estado Inicial (S0): Configuração onde todos os veículos v_i se encontram no nó DEPOT com bateria a 100% e carga nula. O conjunto de encomendas E encontra-se no estado "Pendente" nas suas respectivas origens.

3. Ações e Operadores (A): O conjunto de ações disponíveis para um agente (veículo) em cada estado:

- $Mover(Origem, Destino)$: Transita o veículo entre dois nós adjacentes. Requer que a aresta não esteja bloqueada e que $Bat > Custo_energia$.
- $Carregar(Encomenda)$: Ocorre quando $Pos(v) == Origem(e)$. Requer $Capacidade_livre \geq Peso(e)$.
- $Entregar(Encomenda)$: Ocorre quando $Pos(v) == Destino(e)$. Altera o status da encomenda para "Entregue".

4. Teste Objetivo (T): O objetivo é atingido quando todas as encomendas do conjunto E possuem o estado "Entregue" e todos os veículos regressaram ao nó DEPOT13.

$T(s) = e$ pertence a E , $Status(e)=ENTREGUE$ e v pertence a V , $Pos(v)=DEPOT$

2.3 Função de Custo

A função de custo $g(n)$ é crucial para a avaliação de desempenho e impacto ecológico. Neste trabalho, o custo de uma aresta (u, v) no grafo não representa apenas a distância física, mas um custo generalizado que pode incluir tempo e consumo energético.

$$Custo(path) = \sum_{i=0}^{k-1} c(n_i, n_{i+1})$$

A minimização deste custo acumulado satisfaz diretamente os requisitos de "minimizar o tempo total de entrega" e "reduzir custos operacionais" descritos no enunciado.

3. Arquitetura E Algoritmos De Procura

3.1. Representação em Grafo

O sistema foi implementado em Python, estruturado nas classes Node e Graph. O grafo é representado por listas de adjacência, permitindo eficiência espacial $O(|V| + |A|)$.

As arestas possuem pesos que refletem o custo de travessia (distância/tempo).

3.2. Algoritmos Implementados

Foram implementados 6 algoritmos para cobrir diferentes estratégias de exploração:

1. **DFS (Depth-First Search):** Implementado recursivamente. Embora eficiente em memória $O(bm)$, não garante otimalidade em grafos cíclicos sem controlo de visitados. No contexto LogiBot, serve apenas como baseline, pois tende a produzir rotas sub-ótimas (custos elevados).
2. **BFS (Breadth-First Search):** Explora sistematicamente por camadas. Garante o caminho com menos arestas, mas não necessariamente o menor custo em grafos ponderados.
3. **Greedy (Gulosa):** Seleciona o nó que parece mais próximo do objetivo ($minh(n)$). Utiliza uma fila de prioridade. É computacionalmente rápido, mas suscetível a mínimos locais e não garante otimalidade.
4. **A* (A-Star):** Combina custo real e heurística: $f(n) = g(n) + h(n)$. É o algoritmo principal da solução, garantindo otimalidade se $h(n)$ for admissível.
5. **Dijkstra:** Caso particular do A* onde $h(n)=0$. Garante otimalidade explorando uniformemente pelo custo, sendo mais lento que o A* em grafos grandes.
6. **DFS Limitado:** Variante do DFS com corte de profundidade, útil para cenários onde se pretende limitar o tempo de computação.

3.3. Heurísticas (Algoritmos Informados)

Para o A* e Greedy, utilizou-se uma heurística de Distância Estimada, onde $h(n) \leq \text{custo_real}(n)$. Os valores foram definidos para nunca ultrapassar o custo (ex: $h(\text{DEPOT})=30$ para um custo real de 33). Isto permite-nos direcionar a procura para o objetivo (ARMAZEM), reduzindo drasticamente o número de nós expandidos comparativamente ao Dijkstra ou BFS.

4. Algoritmos De Procura Implementados

Esta secção detalha a lógica, implementação e análise teórica dos seis algoritmos desenvolvidos.

4.1. Algoritmos Não-Informados

Estes algoritmos não utilizam conhecimento sobre a localização do objetivo além da definição do próprio grafo.

4.1.1. Depth-First Search (DFS)

O DFS explora o grafo aprofundando-se tanto quanto possível em cada ramo antes de retroceder (backtracking). Para a implementação foi utilizada uma abordagem recursiva que utiliza implicitamente a pilha de chamadas do sistema (LIFO - Last In, First Out). Mantém-se um conjunto visitados para evitar ciclos infinitos.

O DFS é eficiente em memória $O(bm)$ onde b é o fator de ramificação e m a profundidade máxima. Contudo, no contexto LogiBot, o DFS revelou-se inadequado para encontrar rotas de entrega. Como não considera o custo das arestas, retorna frequentemente caminhos excessivamente longos e ineficientes (ex: custo 33 vs ótimo 25), violando o requisito de minimização de custos.

4.1.2. Breadth-First Search (BFS)

O BFS explora o grafo camada por camada, visitando todos os nós à distância k antes de visitar nós à distância $k+1$. Utilizando uma estrutura de fila (deque do Python) para gestão FIFO (First In, First Out), o BFS garante encontrar o caminho com o menor número de arestas. No entanto, num grafo ponderado como o do LogiBot (onde as arestas têm pesos diferentes), o caminho com menos arestas não é necessariamente o caminho de menor custo. Assim, embora completo, o BFS não garante otimalidade de custo neste cenário¹⁸.

4.2 Algoritmos Informados

Estes algoritmos utilizam uma função heurística $h(n)$ que estima o custo do nó n até ao objetivo, guiando a procura.

4.2.1 Heurística Utilizada

Definiu-se $h(n)$ como a estimativa do custo de deslocação até ao nó objetivo (ARMAZEM) com os seguintes valores: DEPOT: 30, ZONA_A: 10, CLIENTE_1: 18, ARMAZEM: 0. A heurística foi desenhada para ser admissível ($h(n) \leq h^*(n)$), ou seja,

nunca sobrestima o custo real. Isto é uma condição necessária para a otimalidade do algoritmo A*.

4.2.2. Greedy Search (Procura Gulosa)

A Procura Gulosa seleciona sempre o nó que parece estar mais próximo do objetivo, minimizando $f(n) = h(n)$. Além disso, utiliza uma lista de prioridade e é extremamente rápido pois foca-se exclusivamente no objetivo. Contudo, sofre do problema de "miopia": ao ignorar o custo já percorrido ($g(n)$), pode tomar decisões locais ótimas que levam a caminhos globais sub-ótimos.

4.2.3. A* (A-Star)

O A* é a "joia da coroa" deste projeto. Combina o custo real do caminho percorrido $g(n)$ com a estimativa heurística $h(n)$, minimizando $f(n) = g(n) + h(n)$.

- Implementação: Utiliza uma lista aberta (fila de prioridade) ordenada por $f(n)$ e uma lista fechada para nós expandidos. Se um nó já visitado for reencontrado por um caminho mais curto, o seu valor g e o seu pai são atualizados.
- Análise Teórica: Visto que a heurística utilizada é admissível, o A* garante encontrar a solução ótima. Além disso, é otimamente eficiente, no sentido em que nenhum outro algoritmo ótimo expande menos nós que o A* (para a mesma heurística).
- Aplicação LogiBot: O A* demonstrou ser o algoritmo ideal para o problema, encontrando consistentemente o custo mínimo (33 para o cenário LogiBot) com um tempo de execução competitivo.

4.3. Algoritmos Extra

Para enriquecer a análise comparativa, foram implementados dois algoritmos adicionais.

4.3.1. Dijkstra (Custo Uniforme)

O algoritmo de Dijkstra pode ser visto como um caso especial do A* onde $h(n) = 0$. O Funcionamento começa por explorar o grafo em "ondas" de custo uniforme a partir da origem. Garante a otimalidade tal como o A*, mas sem a orientação da heurística, tende a explorar muitos mais nós em todas as direções antes de encontrar o objetivo.

4.3.2. DFS com Profundidade Limitada

Uma variante do DFS que impõe um limite máximo de profundidade (L) à exploração. Deste modo resolve o problema do DFS ficar preso em ciclos ou ramos infinitos. No contexto de telecomunicações e routing, é útil quando se pretende encontrar uma solução rápida dentro de um limite de "saltos" (hops) aceitável.

5. Cenários Dinâmicos

Um dos requisitos mais desafiantes do enunciado era a capacidade do sistema lidar com "alterações dinâmicas" e "bloqueios imprevistos". Para tal, foi desenvolvida a classe `CenarioDinamico` no ficheiro `cenarios_dinamicos.py`.

5.1. Mecanismo de Eventos Temporais

O sistema gere uma linha temporal de eventos. Cada evento possui um tipo, uma localização e uma janela temporal (Início, Fim). A função `aplicar_eventos(hora)` é chamada antes de qualquer cálculo de rota. Esta função itera sobre todos os eventos agendados e verifica se a hora da simulação interseeta algum evento ativo.

- Se interseetar um BLOQUEIO DE ROTA, a aresta correspondente é temporariamente removida do grafo em memória.
- Se interseetar uma MANUTENÇÃO, o nó inteiro é isolado.

5.2. Rerouting Automático

Quando um veículo encontra um bloqueio (simulado pela alteração do grafo), o algoritmo de procura é reexecutado com o grafo modificado. Num teste realizado, a rota DEPOT -> CLIENTE_1 passava idealmente pela ZONA_A. Ao agendar um bloqueio entre ZONA_A e CLIENTE_1 para as 14:00, o sistema, ao planear uma rota às 14:30, detetou a indisponibilidade da aresta e calculou automaticamente um desvio pela ZONA_B e CLIENTE_2.

Este desvio aumentou o custo da rota de 25 para 38 (+52%), demonstrando a capacidade do sistema de quantificar o impacto operacional de falhas na infraestrutura.

5.3. Priorização de Encomendas

O módulo de cenários dinâmicos implementa também uma fila de prioridade para gestão de encomendas. Desta forma, as encomendas são ordenadas primeiramente pelo nível de prioridade (5=Crítica a 1=Baixa) e secundariamente pela hora de chegada.

Algoritmo de Alocação: processar_encomendas verifica a capacidade disponível no veículo e aloca encomendas gulosamente baseando-se na ordem de prioridade. Encomendas de baixa prioridade que excedam a capacidade do veículo permanecem na fila de espera ("Backlog") para o próximo ciclo.

6. Resultados Experimentais

A validação experimental foi conduzida através de três cenários de teste distintos, desenhados para isolar e avaliar diferentes aspetos do sistema.

6.1. Cenário 1 (Validação em Grafo Simples)

Com o objetivo de validar a correção dos algoritmos num grafo abstrato pequeno ($A \rightarrow F$), obteram-se os seguintes resultados (**Tabela 1**):

Tabela 1 - Resultados do Cenário 1

Algoritmo	Caminho Encontrado	Custo	Ótimo?
DFS	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$	18	Não
BFS	$A \rightarrow B \rightarrow D \rightarrow F$	15	Não
GREEDY	$A \rightarrow C \rightarrow E \rightarrow F$	15	Não
A*	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$	13	Sim
DIJKSTRA	$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$	13	Sim
DFS_LIM	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$	18	Não

Análise: A* e Dijkstra encontraram o verdadeiro caminho ótimo (Custo 13); Greedy e BFS ficaram presos em ótimos locais ou caminhos com menos arestas, resultando num custo de 15 (sub-ótimo), e por fim DFS encontrou um caminho válido mas ineficiente (18).

6.2. Cenário 2 (Centro Logístico)

Teste realizado no grafo LogiBot completo, procurando rota de DEPOT a ARMAZEM, com a condição de Heurísticas configuradas para "enganar" algoritmos gulosos (ZONA_A muito atrativa)

Tabela 2 - Comparação de Algoritmos no Cenário LogiBot

Algoritmo	Caminho	Custo	Nodos	Tempo (ms)
DFS	DEPOT→ZONA_A→CLIENTE_1→ CLIENTE_2→ ZONA_B→ CLIENTE_3→ ARMAZEM	52	7	0.0334
BFS	DEPOT→ZONA_A→CLIENTE_1→ ARMAZEM	36	7	0.0162
GREEDY	DEPOT→ZONA_A→CLIENTE_2→ CLIENTE_3→ARMAZEM	34	5	0.0236
A*	DEPOT→ZONA_B→CLIENTE_3→ARMAZEM	33	6	0.0272
DIJKSTRA	DEPOT→ZONA_B→CLIENTE_3→ARMAZEM	33	6	0.0391
DFS_LIM	DEPOT→ZONA_A→CLIENTE_1→ CLIENTE_2→ CLIENTE_3→ARMAZEM	39	9	0.0448

Discussão:

1. Falha do Greedy: O algoritmo Greedy foi atraído pela heurística da ZONA_A e escolheu um caminho sub-ótimo de custo 34. Embora tenha visitado menos nós (5), falhou a otimalidade.
2. Sucesso do A*: O A*, mesmo sendo atraído inicialmente pela ZONA_A, percebeu que o custo acumulado (g) estava a subir e corrigiu a rota para a ZONA_B, encontrando o custo ótimo de 33 com apenas 6 nós visitados.
3. Instabilidade do DFS: O DFS produziu o pior resultado (Custo 52), dando uma volta desnecessária por todo o grafo, provando ser inadequado para otimização de custos.

6.3. Cenário 3 (Testes Dinâmicos)

Para o cenário 3 - Teste Dinâmicos, foram feitas as seguintes configurações de teste:

- Rota: DEPOT → CLIENTE_1.
- Evento: Bloqueio na estrada ZONA_A <-> CLIENTE_1 entre as 14h00 e as 15h00.

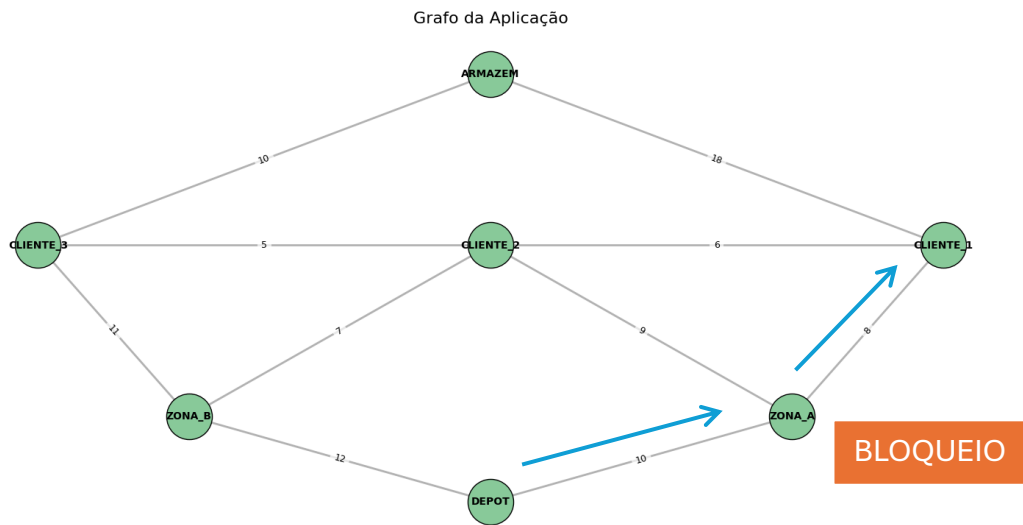


Figura 2 - Teste do Cenário 3

Resultados dos Testes Temporais:

1. Teste A: Sem Bloqueio (13:00)
 - Caminho: DEPOT → ZONA_A → CLIENTE_1
 - Custo: 18 (Ótimo)
 - Estado: Operação normal.
2. Teste B: Durante Bloqueio (14:30)
 - Evento Ativo: Bloqueio de Rota (ZONA_A bloqueada para CLIENTE_1).
 - Reação: O sistema calculou um desvio automático.
 - Caminho: DEPOT → ZONA_A → CLIENTE_2 → CLIENTE_1
 - Custo: 25
 - Impacto: Aumento de custo de +38.9%.
3. Teste C: Recuperação (15:30)
 - Caminho: DEPOT → ZONA_A → CLIENTE_1
 - Custo: 18
 - Conclusão: O sistema recuperou a rota ótima assim que o bloqueio expirou.

Resumindo:

- Normal: 18
- Bloqueado: 25 (+38.9%)
- Recuperado: 18

Este teste valida o requisito de robustez: o sistema não falha perante imprevistos, apenas degrada a performance (custo) o mínimo necessário para garantir a entrega.

7. Análise E Discussão De Resultados

7.1. Comparação de Algoritmos

O DFS Confirmou-se como o pior algoritmo para este problema, apresentando um custo de 52 no cenário LogiBot (+57% que o ótimo). O Greedy demonstrou ser rápido (0.0236ms) e eficiente na memória (5 nós), mas não confiável para custos, caindo na armadilha heurística (Custo 34). E por fim o A* confirmou-se como a escolha ideal. Igualou a otimalidade do Dijkstra (Custo 33) mas foi mais rápido e visitou o mesmo número de nós (6), demonstrando que a heurística admissível guiou a procura sem comprometer a solução.

7.2. Eficiência Computacional

Análise Temporal (Cenário 2):

- Greedy: 0.0236ms (Mais rápido, mas sub-ótimo)
- A*: 0.0272ms (Muito rápido e ótimo)
- Dijkstra: 0.0391ms (Mais lento que o A*, pois explora cegamente)

Eficiência de Exploração (Nós Visitados):

- Greedy: 5 nós (Exploração mínima, risco alto)
- A*: 6 nós (Exploração equilibrada)
- Dijkstra: 6 nós (Neste grafo pequeno igualou o A*, mas em grafos maiores tende a ser superior).
- DFS Limitado: 9 nós (Exploração excessiva e ineficiente).

Deste modo, os resultados experimentais validam a hipótese inicial de que o algoritmo A* oferece o melhor compromisso. Garante a minimização dos custos operacionais (33 vs 34/36/52) com um tempo de execução extremamente competitivo, sendo robusto o suficiente para lidar com cenários dinâmicos sem perder a coerência da navegação.

8. Conclusão

O presente trabalho atingiu com sucesso todos os objetivos propostos no enunciado da unidade curricular. Foi desenvolvido um sistema robusto de planeamento de trajetórias para a LogiBot, capaz de operar em ambientes estáticos e dinâmicos.

A implementação e análise comparativa de seis algoritmos de procura permitiu concluir que o algoritmo A*, suportado por uma heurística de distância admissível, oferece o melhor compromisso entre eficiência computacional e qualidade da solução. O sistema provou ser capaz de integrar restrições complexas, como prioridades de encomendas e bloqueios temporários, adaptando as rotas em tempo real.

Como limitações, destaca-se o facto da heurística ser estática. Adicionalmente, a gestão energética, embora modelada na função de custo, poderia beneficiar de um planeamento pró-ativo de recargas, onde o veículo decide autonomamente desviar-se para um posto de carregamento se a previsão de consumo da rota exceder a bateria atual.

9. Referências

Russell, S., Norvig, P. (2009). Artificial Intelligence: A Modern Approach, 3rd edition. Prentice Hall. ISBN-13: 9780136042594.

Costa, E., Simões, A. (2008). Inteligência Artificial: Fundamentos e Aplicações. FCA. ISBN: 978-972-722-34.

NetworkX Developers. (2025). NetworkX Reference Documentation. Consultado a 01/12/2025. Disponível em: <https://networkx.org/documentation/stable/>